Day 2: Application Containerization

---

1. Dockerfile Deep Dive

Definition

A Dockerfile is a script containing instructions to build a Docker image in a reproducible and automated way.

Key Instructions

Instruction	Purpose
FROM	Base image
WORKDIR	Set working directory
COPY / ADD	Copy files into image
RUN	Execute commands during build
CMD	Default command at runtime
ENTRYPOINT	Fixed runtime command

Example Dockerfile

```
FROM python:3.9

WORKDIR /app

COPY . .

RUN pip install -r requirements.txt

CMD ["python", "app.py"]
```

1.1 Multi-Stage Builds

Definition

Multi-stage builds allow using multiple base images to reduce final image size and improve security.

Example

```
# Stage 1: Build
FROM node:18 AS builder
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
RUN npm run build

# Stage 2: Production
FROM nginx:alpine
COPY --from=builder /app/build /usr/share/nginx/html
```

Benefits

- Smaller image size
- No unnecessary build tools in final image
- Improved security

Scenario

Problem:

- Image size = 1.2 GB
- Contains build tools, source code

Solution:

- Use multi-stage → final image < 200 MB

1.2 Docker Build Caching

Definition

Docker reuses previous build layers to speed up image creation.

Example

Bad practice:

```
COPY . .
RUN npm install
```

Good practice:

```
COPY package.json .
RUN npm install
COPY . .
```

Explanation

- Dependencies change less frequently
- Cache reused → faster builds

Scenario

CI/CD pipeline taking too long

Solution:

- Optimize layer ordering → reduce build time

---

2. Containerizing Monolithic Applications

Definition

Packaging a large, tightly coupled application into a container without breaking it into microservices.

Steps

1. Identify runtime (Node, Java, Python)
2. Create Dockerfile
3. Configure dependencies
4. Expose required ports
5. Build and run container

Example (Node.js App)

```
FROM node:18
WORKDIR /app
COPY package.json .
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Scenario

Legacy Banking Application

- Monolithic Java app deployed on VM

Problem:

- Hard to scale
- Environment inconsistency

Solution:

- Containerize using Docker
- Deploy on Kubernetes

Case Study

Enterprise Migration

- Legacy payroll system containerized
- Initially kept as monolith

Result:

- Faster deployment cycles
- Easier migration to cloud

---

3. Application Environment Configurations

Definition

Externalizing configuration so that the same container can run in different environments (Dev, QA, Prod).

Methods

1. Environment Variables

ENV DB_HOST=localhost

ENV DB_PORT=5432

Run-time:

docker run -e DB_HOST=prod-db myapp

---

2. .env Files

DB_HOST=localhost

DB_PORT=5432

---

3. Config Injection (Best Practice)

- Kubernetes ConfigMaps
- Secrets

Scenario

Same application deployed in:

- Dev → local DB
- Prod → cloud DB

Solution:

- Use environment variables

Case Study

E-commerce Platform

Problem:

- Hardcoded DB credentials

Solution:

- Use environment variables + secrets

Result:

- Secure and flexible deployments

---

4. Debugging Container Builds

Common Issues

1. Build Failures

- Missing dependencies
- Incorrect paths

2. Runtime Errors

- Application crashes
- Port issues

Debugging Techniques

1. Check Build Logs

```
docker build .
```

2. Run Container Interactively

```
docker run -it myapp /bin/bash
```

3. Inspect Logs

```
docker logs <container_id>
```

4. Check Running Containers

```
docker ps
```

Scenario

Application not starting

Steps:

1. Check logs

2. Verify port
3. Validate dependencies

Case Study

Production Failure

Problem:

- Container crashes after deployment

Root Cause:

- Missing environment variable

Solution:

- Add env configuration

Result:

- Stable deployment

---

5. Hands-On: Containerize a Legacy Application

Step 1: Choose Application

Options:

- Node.js app
- Python Flask app
- Java Spring Boot app

Step 2: Create Dockerfile

Example (Python Flask):

```
FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install flask
EXPOSE 5000
```

```
CMD ["python", "app.py"]
```

Step 3: Build Image

```
docker build -t legacy-app .
```

Step 4: Run Container

```
docker run -p 5000:5000 legacy-app
```

Step 5: Optimize Dockerfile

- Use smaller base image (alpine)
- Use multi-stage builds
- Optimize caching

Optimization Example

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["python", "app.py"]
```

End-to-End Case Study (Day 2 Summary)

Scenario: Legacy HR Application

Problem:

- Runs only on specific server
- Deployment takes hours
- Environment mismatch issues

Solution

1. Create Dockerfile
2. Use environment variables
3. Optimize image using multi-stage builds
4. Debug and test container

Result

- Portable application
- Faster deployments
- Consistent environments
- Reduced infrastructure dependency